ENEE 408C: Project Specification

University of Maryland, College Park, `Fall 2025`
Prof. Shuvra S. Bhattacharyya

1 Collaboration Policy

Students will work in teams. Students who are grouped together in the same team are allowed to discuss the project requirements, and to collaborate on all aspects of code development, testing, debugging, and documentation, unless otherwise specified as part of specific requirements for the project. Each team is responsible for a single set of deliverables, which represents the collective effort of the team.

2 Overview

In this project, you will develop an accelerator for polynomial computations called the *polynomial evaluation accelerator* (*PEA*). The overall interface of the accelerator is illustrated in Figure 1. Each of the four buffers shown in Figure 1 represents a dual-ported, first-in, first-out (FIFO) buffer.

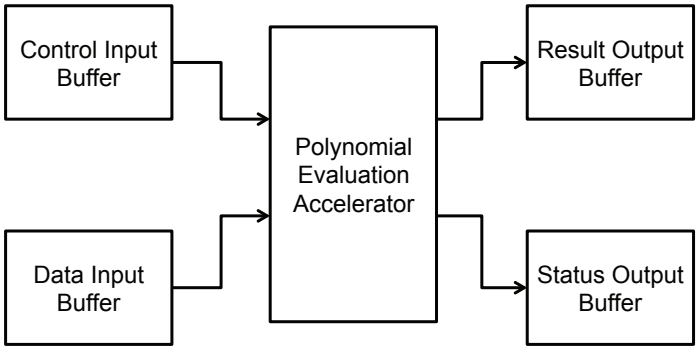

Figure 1: Block diagram of the polynomial evaluation accelerator, along with the buffers that it is required to interface to.

There are two components in the project — a software component, and a hardware (Verilog-based) component. In the software component, you will implement the PEA as a LIDE-C actor, while in the hardware component of the project, you will be required to implement the PEA in Verilog, and leverage the applicable testing-related files from your LIDE-C version of the actor to derive tests for your Verilog-based PEA actor implementation. The functional specification of the PEA actor is provided in Section 3, and Section 4.

Your Verilog implementation should be developed using LIDE-V, which provides integration of lightweight dataflow programming with the Verilog hardware description language. Your Verilog-based implementation can be viewed as an application-specific *accelerator* that is dedicated to polynomial evaluation.

Two issues in Verilog implementation are resource requirements (hardware cost) and performance. For the purposes of this project, *performance* is the average latency for processing streams of instructions and data. Specific streams ("benchmark inputs") that you use to evaluate performance should be designed as part of your experimental approach, and should be documented in your project report.

Trade-offs between resource requirements and performance are often crucial to the hardware design and implementation process. The following section summarizes objectives and requirements associated with

Verilog components of your project, and the associated trade-off evaluation study.

3 System Specification

The following points summarize the functionality of the PEA, and associated design requirements.

1. The PEA operates by repeatedly reading instructions from the control input buffer (see Figure 1), processing these instructions based on any relevant data from the data input buffer, and writing the results and result status information (defined below) to its two output buffers.

2. The PEA maintains 8 sets of polynomial coefficients — each of these 8 coefficient vectors (CVs) corresponds to a different polynomial whose coefficients are cached inside the PEA for rapid evaluation. We label these CVs as $S_0, S_1, \ldots, S_7$. The index of each of these sets is called the "address" of that set — i.e., i is the address of each S_i.

3. Each CV S_i is of the form $S_i = (c_{i,0}, c_{i,1}, \ldots, c_{i,n_i})$, where n_i represents the degree of the polynomial that corresponds to S_i, and each $c_{i,j}$ represents an integer value. We require that $0 \leq n_i \leq 10$; so that the maximum polynomial degree is 10. Note that different S_i's can have different degrees. Given the polynomial P_i that is represented by an S_i, evaluating this polynomial with argument x corresponds to computing the following value

$$P_i(x) = \sum_{k=0}^{n_i} c_{i,k} \times x^k. \tag{1}$$

4. Polynomial arguments (x values) are maintained as 16-bit signed, two's complement integers. Each polynomial coefficient (each $c_{i,k}$) is also maintained in a 16-bit, signed, two's complement format. The results of polynomial evaluation are delivered as 32-bit signed, two's complement integers. There should be no loss of precision if the result of a polynomial evaluated by Equation 1 fits within the specified 32-bit result format. Beyond this, there is no requirement for overflow handling. It is the user's responsibility to ensure freedom from overflow to ensure predictable operation of the PEA on a stream of polynomial computations.

5. Recall that the PEA repeatedly reads and executes instructions from the control input buffer. Each instruction occupies a single word in the control input buffer, which means that it can be read out with a single read (dequeue) operation on the FIFO. There are four PEA instructions: store polynomial (STP), evaluate polynomial (EVP), evaluate block (EVB), and reset (RST). These instructions operate as described in Section 4.

6. As described in Item 4, overflow handling is not required. The status output of the PEA should be used to indicate non-overflow errors, such as a CV that is used in an EVP instruction before it has been set with STP; a polynomial address that is out of range (if your instruction encoding permits this); an invalid number of coefficients; etc. Be as thorough as possible in detecting errors and signaling these errors through the status output. There should be a dedicated status output value that indicates the absence of any error (i.e., an indicator of correct operation), while other status output values correspond to specific kinds of error conditions. Each STP and EVP instruction results in a single status output value, and each EVB instruction results in b status output values, where b is the block size. The RST instruction does not result in any status output.

7. Observe that for every status output value produced, there is a single, corresponding result value that is also produced. If a status output indicates that an error has occurred, then the corresponding result value should be zero. Thus, errors do not result in result outputs being skipped, and furthermore, errors do not result in halting of the system.

8. If the PEA is ready to produce a new output result but the result output buffer or status output buffer is full, then the PEA should effectively stall until there is sufficient space in both output buffers. Similarly, if the PEA is expecting input from one of the input buffers, but that input buffer is empty, then the PEA should stall until sufficient data arrives in that buffer for the PEA to continue. Here, by "stalling" we mean that the enable method returns `FALSE` (for C implementation), and the actor enable module (for HDL implementation) produces a `FALSE` output signal.

9. Note that the four FIFO buffers shown in Figure 1 are external to the PEA — the PEA should be designed to *interface* to a set of four buffers having arbitrary capacities (but fixed word-lengths, which you should specify as part of your design).

4 PEA Instructions

This section summarizes details on the operation of each PEA instruction.

1. The STP instruction has the form STP A N, where A is a CV address (i.e., an integer between 0 and 7, inclusive), and N is a polynomial degree specifier (i.e., an integer between 0 and 10, inclusive). The instruction has the effect of reading $(N+1)$ successive values $x_0, x_1, \ldots, x_N$ from the data input buffer, and setting the CV S_A so that $S_A = (x_0, x_1, \ldots, x_N)$. Thus, S_A is set so that $c_{A,j} = x_j$ for $0 \leq j \leq N$. Any previous setting of S_A is overwritten.

2. The EVP instruction has the form EVP A, where A is a CV address. The instruction has the effect of reading a polynomial argument x from the data input buffer, evaluating the polynomial referenced by A with the argument x, and sending the result and status of this evaluation to the result output buffer and status output buffer, respectively. See Item 4 in Section 3 for the definition of a polynomial argument in this context.

3. The EVB instruction can be viewed as a shorthand for executing successive EVP instructions with the same CV. The EVB instruction has the form EVB A b, where A is a CV address, and b is a non-negative integer between 0 and 31, inclusive. The instruction has the effect of repeatedly — for b iterations — reading a value y from the data input buffer; evaluating the polynomial referenced by A with the operand y; and sending the result and status of this evaluation to the result output buffer and status output buffer, respectively. Each EVB instruction therefore results in b values being read from the data input buffer, where b is the value of the second instruction operand. Each EVB instruction also results in b values being written to each of the two output buffers. If b is invalid, then a single result/status pair should be produced with an error-status indicated. If A is invalid but b is valid, then b result/status pairs should be produced with error-status indication for all pairs.

4. The RST instruction takes no operands. The RST instruction has the effect of "emptying" out the set of CVs so that they are all in an "empty" state. Thus, the effect of all previous STP instructions is lost, and to use any CV again (in an EVP or EVB instruction), the user has to first call STP to store new coefficients in the CV. The RST instruction also has the effect of canceling any instructions in the PEA that are in the middle of execution. These are instructions that have been read from the control input buffer already, but have not yet been executed to completion.

5 Design Structure

You have considerable freedom in this project on how to structure your design of the PEA. This flexibility will enable you to experiment extensively with different design options in a manner that permits exploration of trade-offs between performance and resource costs.

6 Objectives and Requirements

1. For the software part of the project, implement the PEA actor using LIDE-C. Your actor design should involve at least one distinct core functional dataflow (CFDF) mode for each of the four PEA instructions — STP, EVP, EVB, RST. You may also incorporate one or more additional modes into your design. You are required to apply unit testing on the PEA actor with *three* distinct tests for each of the core processing instructions (STP, EVP, and EVB); one test for the RST instruction; and *five* distinct tests that exercise the actor operating on streams that involve multiple instructions and their corresponding operands.

2. The overall PEA design should satisfy CFDF semantics. That means that it should be designed in terms of a set of modes (or parameterized sets of modes) where each mode has fixed production and consumption behavior.

3. For the hardware part of your project, all parts of your PEA design *should be developed using synthesizable Verilog* and synthesized using the Vivado synthesis tool. You will use the synthesis reports generated by Vivado to evaluate your design in terms of processing speed and resource utilization. Note that the input/output buffers shown in Figure 1 are not considered to be part of the PEA, and do not need to be developed using synthesizable Verilog.

4. A major objective in the hardware part of the project is for you to explore different designs for the PEA along with their trade-offs in terms of resource requirements and performance. You should summarize your findings in a project report. Document in your report each design that you explored and what you learned from it — e.g., what is its efficiency in terms of resource requirements and performance; what, if any, improvements did you discover for the design as you developed, experimented with, and analyzed it; how efficient is the design from a design complexity point of view (i.e., in terms of having an implementation that is easy to understand and maintain)? Additional guidelines and requirements for the project report will be given in a separate handout.

5. In your report, highlight the Pareto designs that you implemented for your design exploration. Provide the performance and resource requirements for these designs, and plot these values on a resource-requirements-versus-performance plot for your set of Pareto designs. Also include a brief discussion of non-Pareto (dominated) designs that you evaluated, and how they compared to the Pareto designs in terms of the relevant metrics. Part of your grade will be based on the quality and diversity of your Pareto designs, as well as on your best design in terms of performance, and your best design in terms of minimizing resource requirements.

6. Discuss the benchmark inputs (see Section 2) that you have used to evaluate and compare the performance of candidate designs. Explain why this set of benchmark inputs is effective at evaluating the efficiency of your design.

7. In your report, you should not limit the discussion to the Verilog part or to just data (performance/area) analysis. You may also discuss aspects like the overall design structure (Verilog and C parts), and the key FSM structures that you designed and how they operate.

8. Document also in your report all relevant aspects of your design that are not already specified in the project specification — e.g., the instruction encoding format of instruction words in the control input buffer, state diagrams for any FSMs used, etc.

9. Submit all of the designs that you implemented, including complete Verilog source code, documentation, and tests to demonstrate correct operation of the code, and tests to demonstrate the performance results reported for your design. These designs should be integrated within the overall project submission directory archive (`polyacc-project.tar.gz`) specified in Section 8. The submission of your Verilog code should be documented with `README.txt` files so that one can easily reproduce and verify your design evaluation results (Pareto results for resource requirements versus performance) by following the directions provided in your documentation.

7 Extra Credit

The project has an optional extra credit component. For extra credit, create an extended-functionality version of the PEA called PEA+, which is augmented with one or more additional instructions. In other words, the "standard" version of the PEA provides four instructions — STP, EVP, EVB, and RST — and the "plus" version provides these four instructions as well as one or more additional instructions. It is up to you to define what these additional instructions will be, what (if any) operands they will have, and how the additional instructions will be simulated (in LIDE-C) and implemented (in LIDE-V).

All of the requirements and objectives described in Section 6 should be followed in your design, documentation, implementation, benchmarking, testing, and reporting of the PEA+. So your submitted `polyacc-project.tar.gz` directory archive should contain all of the designs that you implemented for the base version, and also all of the designs that you implemented for the plus version.

Similarly, your project report should highlight your Pareto designs for the base version and also for the plus version. Your project report should also provide comparisons — both qualitative and quantitative — of the achieved performance/resource-requirements trade-offs for your base and plus versions of the PEA.

8 Deliverables

Except for the project report, the deliverables for the project must be archived, using `dxpack`, in an archive file called `polyacc-project.tar.gz`, where `polyacc` stands for "polynomial evaluation accelerator". The project report should be submitted on Canvas. More details about the expected contents and form of the deliverables will be specified in a separate handout.

The archive file `polyacc-project.tar.gz` must be submitted by 11:30PM on Friday, December 12, 2025.

Document Version: Last updated on October 19, 2025.